Smart Inventory Management System for Construction Sites

1. Project Overview

The Smart Inventory Management System for Construction Sites is an IoT-based project developed to automate the monitoring and management of construction materials. Construction sites require different types of materials such as cement, sand, bricks, steel rods, aggregates, and other components. Maintaining accurate records of these materials through manual inspection is difficult and time-consuming. The proposed system uses sensors, an ESP8266 microcontroller, display units, and alert devices to continuously monitor the availability of materials.

A load cell can be used to measure the weight of materials, while an ultrasonic sensor can measure the level of materials stored inside a container. The ESP8266 processes the sensor information and determines the current inventory condition.

The information can be displayed on an LCD and transmitted through Wi-Fi for remote monitoring. When the material quantity falls below a predefined minimum level, the system activates a warning LED and buzzer. This system helps reduce manual work, prevents material shortages, reduces wastage, and improves overall inventory management at construction sites.

2. Problem Statement

Construction sites involve the storage and continuous usage of large quantities of materials. Proper management of these materials is essential to ensure that construction activities are completed without interruption. In conventional construction sites, workers generally inspect the available stock manually and maintain records using notebooks or spreadsheets.

This method requires considerable time and may result in inaccurate inventory information due to human errors. Materials may also be consumed without being recorded properly, making it difficult to determine the actual quantity available. If essential materials such as cement, sand, or steel become unavailable unexpectedly, construction activities may be delayed.

On the other hand, purchasing excessive quantities can increase storage requirements, material wastage, and project costs. Materials stored in different areas of a large construction site are also difficult to monitor regularly. Therefore, there is a need for an automated system that can continuously measure material quantities, identify low-stock conditions, and provide timely alerts to construction-site personnel.

T

3. Proposed Solution

The proposed Smart Inventory Management System provides an automated solution for monitoring construction materials. The system uses a load cell and HX711 module to determine the weight of materials stored in a container

as the main control unit of the system. The controller processes the sensor readings and compares the measured quantity with a predefined minimum stock level. When sufficient material is available, the system indicates a normal condition using a green LED. When the quantity decreases below the specified limit, the red LED is activated and a buzzer produces an audible warning.

An LCD can display the current material quantity and stock condition. Since the ESP8266 has Wi-Fi capability, the system can also be connected to an IoT platform for remote monitoring. This solution provides faster inventory updates, reduces manual inspection, and helps construction workers take timely action when materials need to be replenished.

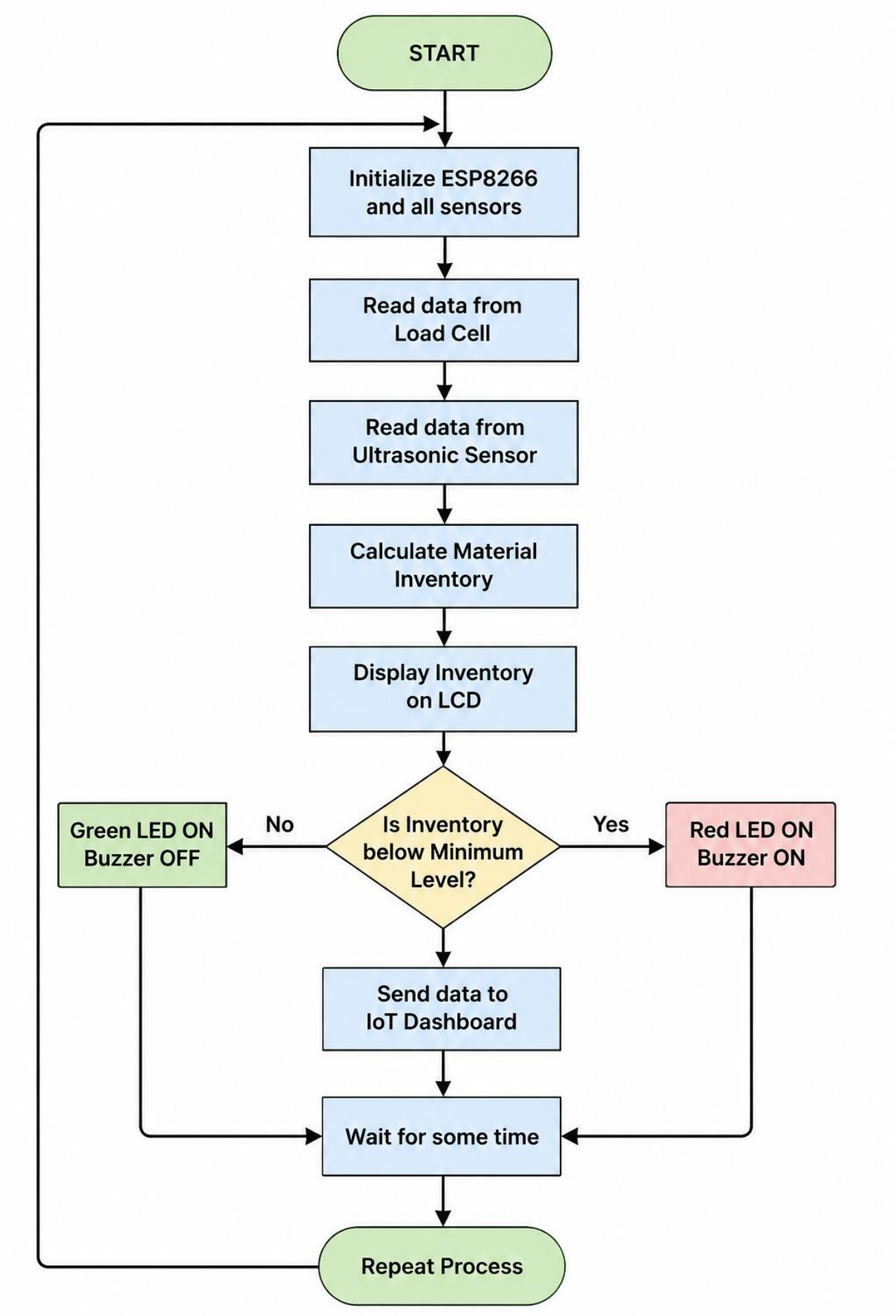
4. System Architecture

The **Smart Inventory Management System for Construction Sites** consists of five main sections: **sensing unit, processing unit, display unit, alert unit, and IoT communication unit**. The sensing unit collects information about the quantity of construction materials using a load cell and ultrasonic sensor. The load cell measures the weight of materials stored in the container, while the ultrasonic sensor measures the material level. The HX711 module amplifies and converts the load-cell signal into digital data.

The **ESP8266 Node MCU** acts as the main processing unit. It receives the sensor data, processes the readings, and compares the available inventory with the predefined minimum stock level. The processed information is sent to the LCD for displaying the current material quantity and status. LEDs provide visual indications, where the green LED represents sufficient stock and the red LED represents low stock. A buzzer provides an audible warning when the inventory reaches the critical level. The ESP8266 can also use its built-in Wi-Fi connectivity to send inventory information to an IoT dashboard for remote monitoring.

Architecture Flow

Material Storage → Sensors → HX711 → ESP8266 → Data Processing → LCD/LED/Buzzer → Wi-Fi → IoT Dashboard


START
Initialize ESP8266 and all sensors
Read data from Load Cell
Read data from Ultrasonic Sensor
Calculate Material Inventory
Display Inventory on LCD
Is Inventory below Minimum Level?
No
Yes
Green LED ON Buzzer OFF
Red LED ON Buzzer ON
Send data to IoT Dashboard
Wait for some time
Repeat Process

5. Circuit Table

Component	Pin	ESP8266 Pin	Function
Load Cell	Signal	HX711	Measures material weight
HX711	DT	D5	Data communication
HX711	SCK	D6	Clock signal
Ultrasonic Sensor	TRIG	D1	Sends ultrasonic pulse
Ultrasonic Sensor	ECHO	D2	Receives reflected pulse
Green LED	Anode	D7	Normal stock indication
Red LED	Anode	D8	Low-stock indication
Buzzer	Positive	D0	Warning alarm
LCD	SDA	D3	Data communication
LCD	SCL	D4	Clock communication
Components	GND	GND	Common ground

Circuit Description

- The load cell is connected to the HX711 module.
- The HX711 module is connected to the digital pins of the ESP8266.
- The ultrasonic sensor is connected to two GPIO pins.
- The LCD is connected through the I2C communication interface.
- LEDs are connected through suitable current-limiting resistors.
- The buzzer is connected to a digital output pin.
- All components share a common ground.
- The power supply must be selected according to the voltage requirements of each component.

6. Circuit Design

The **ESP8266** acts as the main controller. The load cell with HX711 measures the weight of stored material, while the ultrasonic sensor measures the material level inside a container.

The ultrasonic sensor is installed at the top of the storage container. It measures the distance between the sensor and the surface of the material. As the material level decreases, the measured distance increases.

The ESP8266 compares the measured quantity with the predefined minimum stock value.

7. Algorithm

The operation of the system begins with the initialization of the ESP8266 and all connected sensors and output devices. After initialization, the load cell measures the weight of the stored material and the ultrasonic sensor measures the material level. The ESP8266 receives these readings and processes them to determine the available inventory. The measured value is then compared with the predefined minimum stock level.

The inventory information can then be displayed on the LCD and transmitted through Wi-Fi. After completing one monitoring cycle, the system waits for a short period and repeats the same process continuously.

If the quantity is above the minimum limit, the system considers the inventory to be normal and turns ON the green LED while keeping the buzzer OFF. If the quantity falls below the minimum level, the system identifies a low-stock condition and turns ON the red LED and buzzer.

8. Coding

The ESP8266 program continuously reads the sensor values and determines the inventory condition.

```
        #include <SPI.h>
#include <MFRC522.h>
#include <ESP8266WiFi.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"
```

```
// ====================================================
// WIFI
// ====================================================

#define WIFI_SSID     "Pradeep"
#define WIFI_PASSWORD "12345678"

// ====================================================
// ADAFRUIT IO
// ====================================================

#define AIO_SERVER      "io.adafruit.com"
#define AIO_SERVERPORT  1883

#define AIO_USERNAME    "muthumkt"
#define AIO_KEY         "aio_Dpts45gXpVu4he3HHcwUXVBgOuKb"

// ====================================================
// RFID PINS
// ====================================================

#define SS_PIN  D8
#define RST_PIN D0

// ====================================================
// 3-PIN BUZZER
// S / SIG -> D3
// +       -> 3.3V
```

```
// -       -> GND
// =======================================================

#define BUZZER_PIN D3

// Active LOW buzzer
#define BUZZER_ACTIVE_LOW true

// =======================================================
// RFID
// =======================================================

MFRC522 rfid(SS_PIN, RST_PIN);

// =======================================================
// WIFI + MQTT
// =======================================================

WiFiClient client;

Adafruit_MQTT_Client mqtt(
  &client,
  AIO_SERVER,
  AIO_SERVERPORT,
  AIO_USERNAME,
  AIO_KEY
);

// =======================================================
```

```
// ADAFRUIT IO FEEDS
// =================================================

Adafruit_MQTT_Publish rfidFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/rfid-card"
  );

Adafruit_MQTT_Publish itemFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/item-name"
  );

Adafruit_MQTT_Publish statusFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/item-status"
  );

// =================================================
// RFID TAG UIDs
// Replace these with your actual RFID UIDs
// =================================================

byte tag1[] = {0xDE, 0xAD, 0xBE, 0xEF};
byte tag2[] = {0x11, 0x22, 0x33, 0x44};
byte tag3[] = {0x55, 0x66, 0x77, 0x88};
```

```
// ====================================================
// ITEM STATUS
// false = IN
// true  = OUT
// ====================================================

bool item1Status = false;
bool item2Status = false;
bool item3Status = false;

// ====================================================
// BUZZER ON
// ====================================================

void buzzerON()
{
  if (BUZZER_ACTIVE_LOW)
  {
    digitalWrite(BUZZER_PIN, LOW);
  }
  else
  {
    digitalWrite(BUZZER_PIN, HIGH);
  }
}

// ====================================================
// BUZZER OFF
```

```
// ====================================================

void buzzerOFF()
{
  if (BUZZER_ACTIVE_LOW)
  {
    digitalWrite(BUZZER_PIN, HIGH);
  }
  else
  {
    digitalWrite(BUZZER_PIN, LOW);
  }
}

// ====================================================
// WIFI CONNECTION
// ====================================================

void connectWiFi()
{
  Serial.println();
  Serial.print("Connecting to WiFi: ");
  Serial.println(WIFI_SSID);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
```

```
    Serial.print(".");
  }

  Serial.println();
  Serial.println("WiFi Connected!");

  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}

// ====================================================
// ADAFRUIT IO CONNECTION
// ====================================================

void connectMQTT()
{
  if (mqtt.connected())
  {
    return;
  }

  Serial.println("Connecting to Adafruit IO...");

  int8_t ret;

  while ((ret = mqtt.connect()) != 0)
  {
    Serial.print("MQTT Error: ");
    Serial.println(mqtt.connectErrorString(ret));
```

```
    mqtt.disconnect();

    delay(5000);
  }

  Serial.println("Adafruit IO Connected!");
}

// =================================================
// SETUP
// =================================================

void setup()
{
  Serial.begin(115200);

  pinMode(BUZZER_PIN, OUTPUT);
  buzzerOFF();

  SPI.begin();

  rfid.PCD_Init();

  delay(1000);

  connectWiFi();

  connectMQTT();
```

```
  Serial.println();
  Serial.println("======================================");
  Serial.println(" SMART INVENTORY MANAGEMENT SYSTEM");
  Serial.println("======================================");
  Serial.println();
  Serial.println("RFID Reader : READY");
  Serial.println("Buzzer      : READY");
  Serial.println("WiFi        : CONNECTED");
  Serial.println("Adafruit IO : CONNECTED");
  Serial.println();
  Serial.println("Waiting for RFID card...");
  Serial.println();
}

// ================================================
// MAIN LOOP
// ================================================

void loop()
{
  // Check WiFi
  if (WiFi.status() != WL_CONNECTED)
  {
    connectWiFi();
  }

  // Check Adafruit IO
  connectMQTT();
```

```
// ================================================
// RFID CARD DETECTION
// ================================================

if (rfid.PICC_IsNewCardPresent())
{
  if (rfid.PICC_ReadCardSerial())
  {
    // Card detected
    buzzerON();

    Serial.println();
    Serial.println("======================================");
    Serial.println("          RFID CARD DETECTED");
    Serial.println("======================================");

    // ================================================
    // CREATE UID STRING
    // ================================================

    String uidString = "";

    for (byte i = 0; i < rfid.uid.size; i++)
    {
      if (rfid.uid.uidByte[i] < 0x10)
      {
        uidString += "0";
      }
```

```
  uidString += String(
    rfid.uid.uidByte[i],
    HEX
  );

  if (i < rfid.uid.size - 1)
  {
    uidString += ":";
  }
}

uidString.toUpperCase();

// ================================================
// DISPLAY CARD DETAILS
// ================================================

Serial.print("Card UID     : ");
Serial.println(uidString);

// ================================================
// TAG 1
// ================================================

if (compareUID(tag1))
{
  Serial.println("Item         : SAFETY HELMET");
```

```
  if (item1Status == false)
  {
    Serial.println("Status       : OUT");

    item1Status = true;

    publishData(
      uidString,
      "SAFETY HELMET",
      "OUT"
    );
  }
  else
  {
    Serial.println("Status       : IN");

    item1Status = false;

    publishData(
      uidString,
      "SAFETY HELMET",
      "IN"
    );
  }
}

// ================================================
// TAG 2
// ================================================
```

```
else if (compareUID(tag2))
{
  Serial.println("Item         : POWER TOOL");

  if (item2Status == false)
  {
    Serial.println("Status       : OUT");

    item2Status = true;

    publishData(
      uidString,
      "POWER TOOL",
      "OUT"
    );
  }
  else
  {
    Serial.println("Status       : IN");

    item2Status = false;

    publishData(
      uidString,
      "POWER TOOL",
      "IN"
    );
  }
```

```
}

// ================================================
// TAG 3
// ================================================

else if (compareUID(tag3))
{
  Serial.println("Item         : DRILL MACHINE");

  if (item3Status == false)
  {
    Serial.println("Status       : OUT");

    item3Status = true;

    publishData(
      uidString,
      "DRILL MACHINE",
      "OUT"
    );
  }
  else
  {
    Serial.println("Status       : IN");

    item3Status = false;

    publishData(
```

```
      uidString,
      "DRILL MACHINE",
      "IN"
    );
  }
}

// ================================================
// UNKNOWN CARD
// ================================================

else
{
  Serial.println("Item         : UNKNOWN");
  Serial.println("Status       : NOT REGISTERED");

  publishData(
    uidString,
    "UNKNOWN",
    "NOT REGISTERED"
  );
}

Serial.println("=====================================");
Serial.println();

// Stop RFID communication
rfid.PICC_HaltA();
rfid.PCD_StopCrypto1();
```

```
    }
  }
  else
  {
    // No card detected
    buzzerOFF();
  }

  delay(100);
}

// ====================================================
// COMPARE UID
// ====================================================

bool compareUID(byte storedUID[])
{
  if (rfid.uid.size != 4)
  {
    return false;
  }

  for (byte i = 0; i < 4; i++)
  {
    if (rfid.uid.uidByte[i] != storedUID[i])
    {
      return false;
    }
  }
```

```
  return true;
}

// ====================================================
// SEND DATA TO ADAFRUIT IO
// ====================================================

void publishData(
  String uid,
  const char* item,
  const char* status
)
{
  Serial.println();
  Serial.println("Sending to Adafruit IO...");

  bool uidSent = rfidFeed.publish(uid.c_str());
  bool itemSent = itemFeed.publish(item);
  bool statusSent = statusFeed.publish(status);

  if (uidSent)
  {
    Serial.println("RFID UID     : SENT");
  }
  else
  {
    Serial.println("RFID UID     : FAILED");
  }
```

```
  if (itemSent)
  {
    Serial.println("Item Name    : SENT");
  }
  else
  {
    Serial.println("Item Name    : FAILED");
  }

  if (statusSent)
  {
    Serial.println("Item Status  : SENT");
  }
  else
  {
    Serial.println("Item Status  : FAILED");
  }

  Serial.println();
  Serial.println("Adafruit IO update complete.");
}
```

9. System Working

The system starts by initializing the ESP8266 and all connected sensors. The load cell continuously measures the weight of the material. The HX711 converts the small electrical signal from the load cell into a readable digital value.

The ultrasonic sensor can additionally monitor the material level inside the storage container. The ESP8266 processes these readings and determines the available inventory.

When the inventory is above the minimum threshold, the system indicates a normal condition using the green LED. When the inventory falls below the threshold, the red LED and buzzer are activated. The inventory information can also be transmitted through Wi-Fi for remote monitoring.

10. Bill of Materials

S. No	Component	Qty.
1	ESP8266 Node MCU	1
2	Load Cell	1
3	HX711 Module	1
4	HC-SR04 Ultrasonic Sensor	1
5	16×2 I2C LCD	1
6	Green LED	1
7	Red LED	1
8	Buzzer	1
9	Resistors	2
10	Breadboard	1
11	Jumper Wires	As required
12	5V Power Supply	1
13	Storage Container	1

11. Prototype Implementation

A small-scale construction-material storage model is developed to demonstrate the proposed system. The storage container represents a material warehouse.

The load cell is positioned underneath the container to measure material weight. The ultrasonic sensor is fixed at the top of the container to determine the material level. The ESP8266 is mounted on a prototype board along with the LCD, LEDs, and buzzer.

Different quantities of material are added and removed during testing. The system detects these changes and updates the inventory condition automatically.

12. Results & Performance Analysis

The prototype demonstrates that construction-material inventory can be monitored automatically without continuous manual inspection.

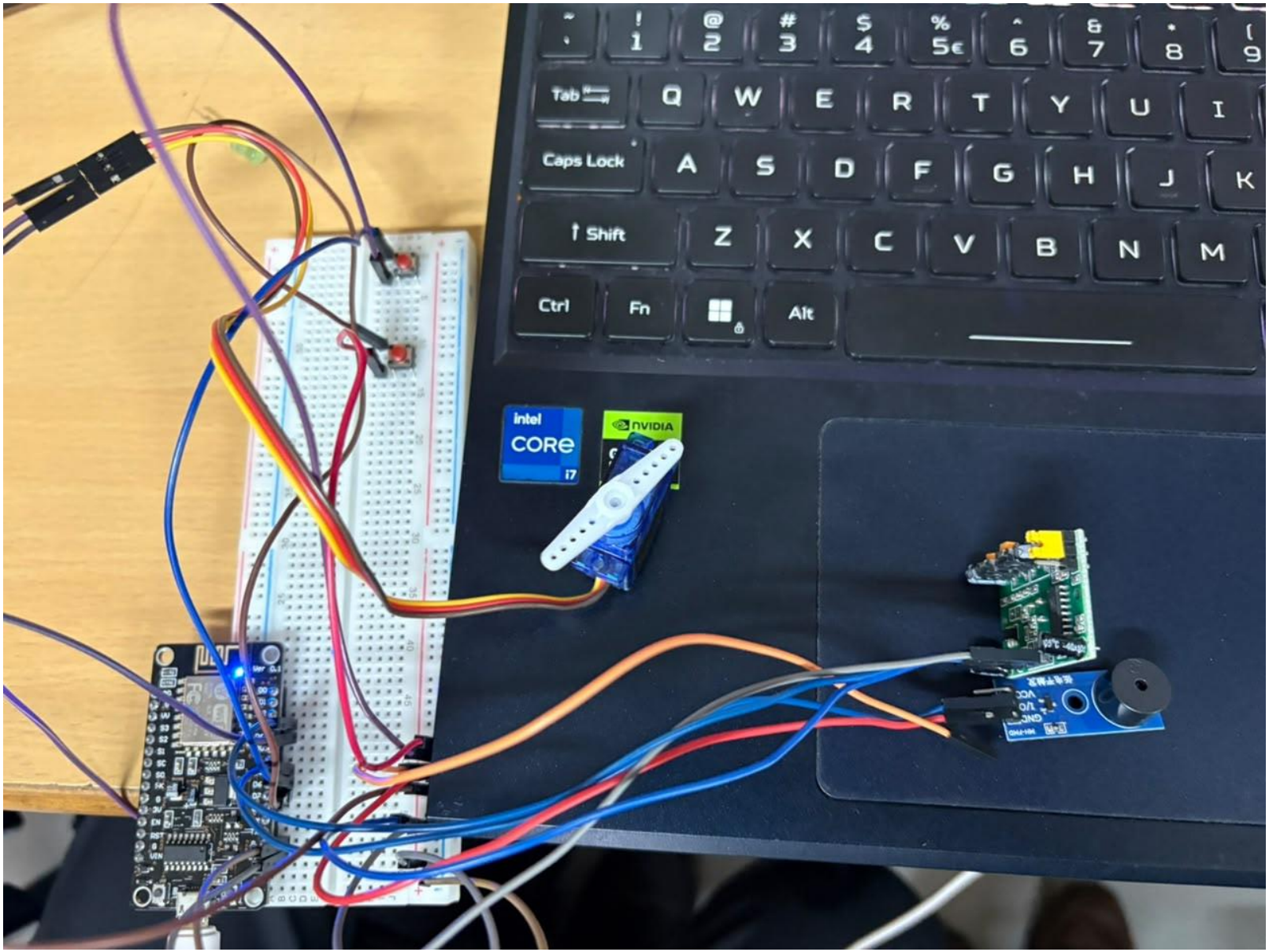
Test Condition	System Response
High material quantity	Green LED ON
Normal material quantity	Green LED ON
Low material quantity	Red LED ON
Critical stock	Buzzer ON
Material added	Inventory value increases
Material removed	Inventory value decreases
Sensor monitoring	Continuous
Remote monitoring	Possible through Wi-Fi

Overall result: The proposed system improves inventory visibility, reduces manual monitoring, provides early low-stock warnings, and can help construction-site personnel avoid material shortages and unnecessary material purchases.

OUTPUT:

```
Sending to Adafruit IO...
RFID UID     : SENT
Item Name    : SENT
Item Status  : SENT

Adafruit IO update complete.
====================================


====================================
          RFID CARD DETECTED
====================================
Card UID     : 1F:5E:EC:48
Item         : UNKNOWN
Status       : NOT REGISTERED

Sending to Adafruit IO...
RFID UID     : SENT
Item Name    : SENT
Item Status  : SENT

Adafruit IO update complete.
====================================


====================================
          RFID CARD DETECTED
====================================
Card UID     : 1F:5E:EC:48
Item         : UNKNOWN
Status       : NOT REGISTERED

Sending to Adafruit IO...
RFID UID     : SENT
Item Name    : SENT
Item Status  : SENT
```


FACULTY ROOM


Caps Lock
Shift
Ctrl
Fn
Alt
intel
CORe
i7
NVIDIA